

Project Report

Implementation of Maximum Subarray Sum using Pipelined Hardware Architecture

(Kadane's Algorithm using Verilog HDL)

Group Name: Kadane's Group

Members:

- Manieesh — CS24B1005
- Goddu S S S S Suraj Sharwan — CS24B1059
- Jeevan Pranav— CS24B1056

Course: Computer Organization and Architecture

Abstract

- This project details the hardware implementation of the Maximum Subarray Sum problem utilizing Verilog Hardware Description Language (HDL).
- While Kadane's Algorithm is traditionally executed as a sequential software program with linear $O(n)$ time complexity, this project successfully translates its algorithmic logic into a synchronous, clock-driven digital architecture.
- The system is fundamentally decoupled into three primary modules to adhere to best practices in ASIC/FPGA design: a Control Finite State Machine (FSM), a Synchronous Memory ROM, and a Pipelined Datapath.
- The FSM operates as the master sequencer, managing memory read timings and pipeline flush cycles, while the datapath autonomously computes the running local sum and compares it against the global maximum.
- Through the implementation of a multi-stage pipeline, the architecture achieves a high processing throughput, allowing it to evaluate one array element per clock cycle after the initial pipeline latency.
- The entire Register Transfer Level (RTL) design was compiled, simulated, and successfully verified using Icarus Verilog and GTKWave, confirming correct signed arithmetic handling and hazard-free data propagation.

Aim of the Project

- **Algorithmic Translation:** To physically implement Kadane's software algorithm in hardware, demonstrating a comprehensive understanding of how high-level programmatic logic is synthesized into RTL modules using Verilog HDL.
- **Architectural Modularity:** To design a strict boundary between the control path and the datapath. This separation ensures the FSM handles pure sequencing without being burdened by arithmetic, allowing for a cleaner, highly scalable design.
- **Performance Optimization:** To apply structural pipelining concepts within the datapath, ensuring that multiple arithmetic operations (accumulation and comparison) can overlap across distinct clock cycles, thereby maximizing the system's theoretical operating frequency.
- **Verification and Debugging:** To utilize industry-standard waveform simulation tools to verify timing constraints, debug synchronous state transitions, and validate that the expected mathematical output perfectly matches the simulated circuit output.

Theory and Mathematical Background

- **The Problem Statement:** The Maximum Subarray Sum problem requires finding the specific contiguous segment within a one-dimensional array of signed integers that yields the absolute largest sum.
- **Input Example:** Given the test array `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`, the circuit must bypass the negative initial values to isolate the optimal segment.
- **Expected Output:** The optimal contiguous subarray is `[4, -1, 2, 1]`, which yields a maximum overall sum of `6`.
- **Kadane's Recurrence Relation:** The hardware datapath is built around executing two simultaneous recurrence equations at every clock cycle:
 - `current_sum = max(current_element, current_sum + current_element)`
 - `max_sum = max(max_sum, current_sum)`
- **Hardware Efficiency:** By deciding at each element whether to extend the previous running sum or start a completely new subarray, the design completely avoids the heavy memory overhead of 2D Dynamic

Programming approaches, making it highly resource-efficient for hardware synthesis.

System Architecture

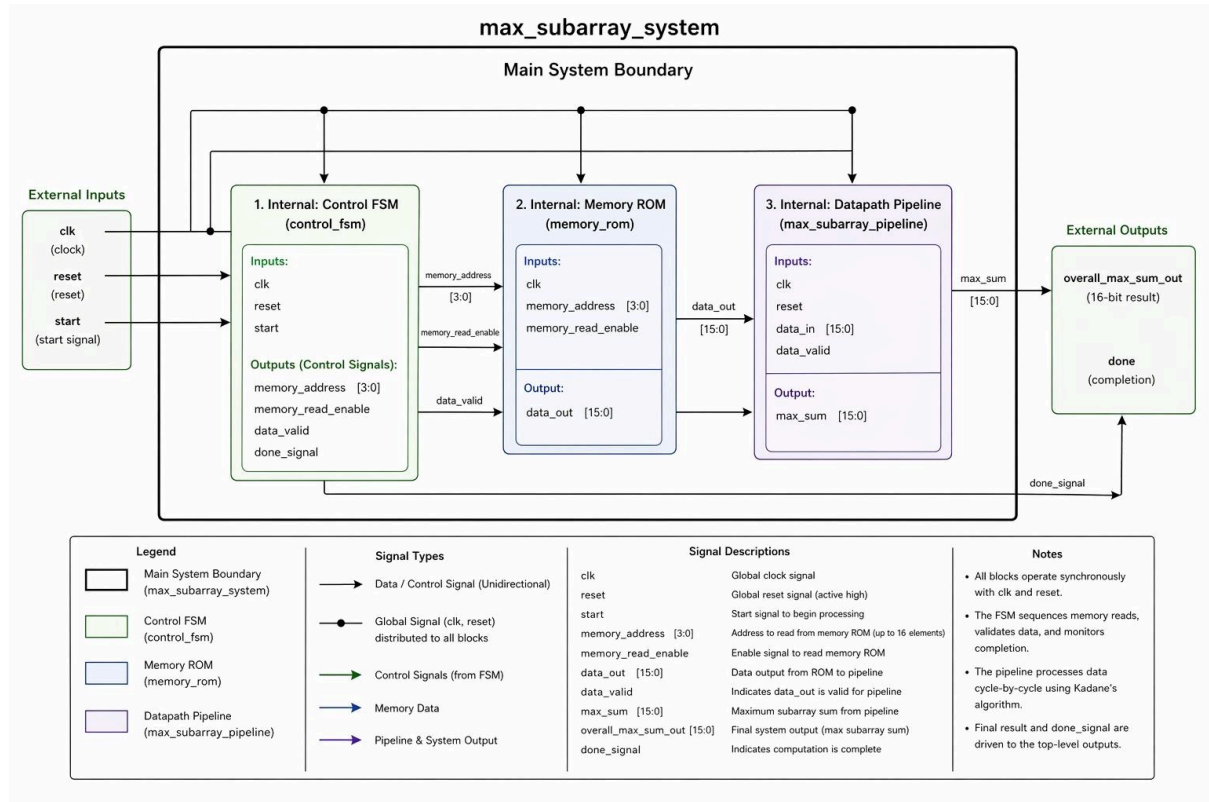


Figure 1: Top-Level System Architecture showing decoupling of Control and Datapath.

- Control FSM:** The "brain" of the operation. It acts as a synchronous sequencer that generates physical memory addresses, drives the ROM read-enable pins, tracks the number of elements processed, and ultimately asserts the final completion signal to the top-level observer.
- Memory ROM:** A synthesized block of read-only memory initialized with the signed test vectors. Operating synchronously, it outputs exactly one 16-bit integer corresponding to the FSM's address pointer at every positive clock edge.
- Datapath Pipeline:** The execution unit containing Arithmetic Logic Units (ALUs) and D-Flip Flops. It ingests the signed integers, executes Kadane's mathematical logic via internal comparators, and buffers intermediate values to maintain a high-speed data stream.

- **Structural Top Module:** The integration layer that wires the Control FSM, ROM, and Datapath together, ensuring that global signals like `clk` and `reset` are distributed simultaneously to prevent race conditions.
- **Testbench Environment:** A dedicated simulation module that provides the external hardware stimulus, including the high-frequency clock toggle, the asynchronous reset pulse, and the `$dumpvars` command to record signal states for visual waveform analysis.

Finite State Machine (FSM) Design

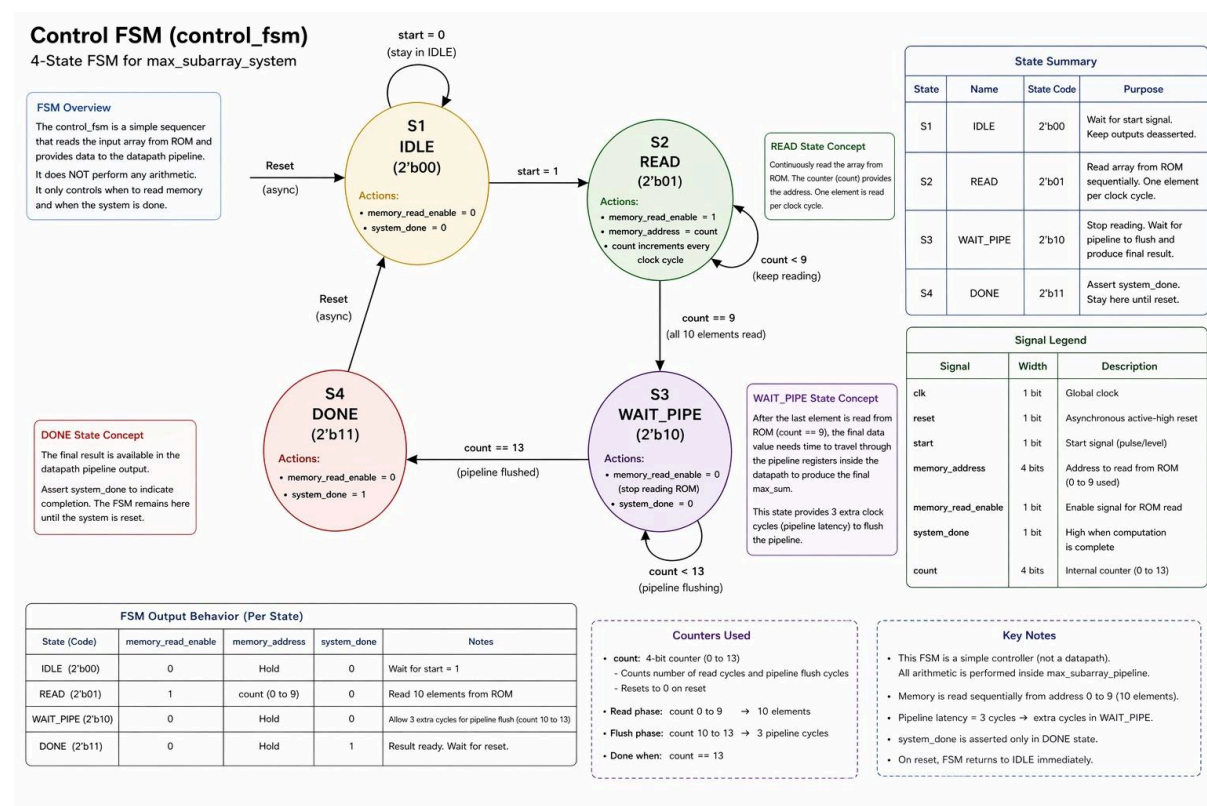


Figure 2: 4-State Control FSM Transition Diagram.

- **State 00 — IDLE:** The default initialization state. The FSM forces all memory read enables low and holds the system in standby until an external, active-high `start` pulse is detected, preventing garbage data from flooding the pipeline.
- **State 01 — READ:** The active data-fetching state. The FSM sequentially increments an internal counter every clock cycle. This counter directly drives the ROM address bus, ensuring all array elements are streamed into the datapath one-by-one.

- **State 10 — WAIT_PIPE:** A critical latency-handling state. Because the datapath is pipelined, the final arithmetic calculation takes longer to process than the final memory read. This state holds the FSM active for three additional clock cycles to allow the "tail" of the data to successfully flush through the internal registers.
- **State 11 — DONE:** The terminal state. Once the pipeline has been fully flushed and the global maximum sum is completely stabilized on the output bus, the FSM asserts the `system_done` flag and locks until a master reset is triggered.

Pipelined Datapath and Hazard Mitigation

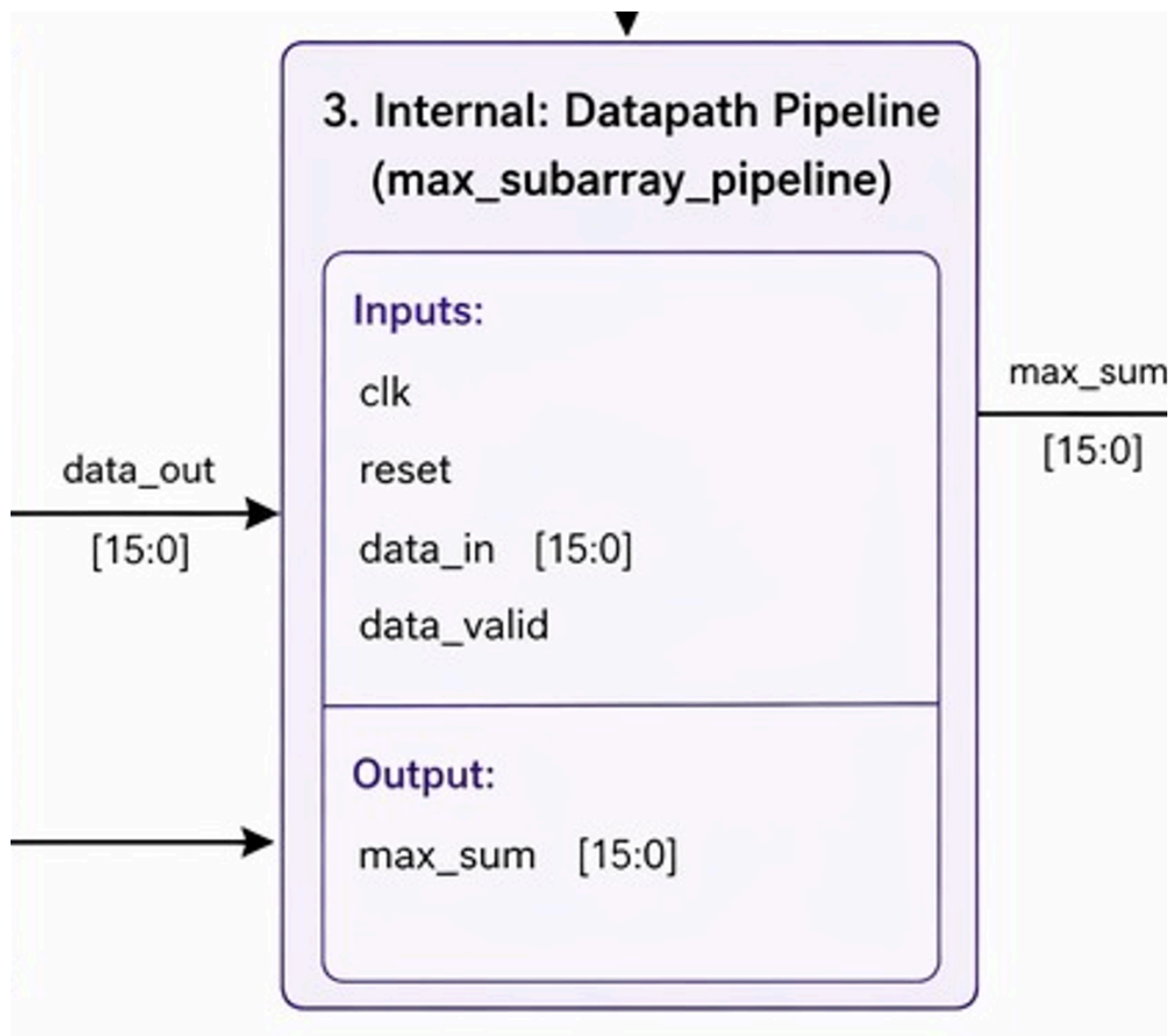


Figure 3: Internal Pipeline Datapath Architecture.

- **Stage 1 (Data Acquisition):** Input data from the Memory ROM is captured by the datapath and aligned with the FSM's `data_valid` flag to ensure arithmetic only occurs on valid integer streams.
- **Stage 2 (Local Accumulation):** The datapath determines whether adding the new integer to the existing running sum is mathematically optimal, or if the current integer is larger than the accumulated sum (triggering a subarray reset).
- **Stage 3 (Global Update):** The newly calculated local sum is compared against the historical global maximum. If the local sum is strictly greater, the global maximum register is updated.
- **Identification of RAW Data Hazard:** During early simulation phases, a critical Read-After-Write (RAW) timing hazard was discovered. The strict one-cycle pipeline delay on the `current_sum` register caused the datapath to read stale accumulation data. When the array transitioned abruptly from a positive running total to a negative integer, the sequential delay corrupted the math.
- **Combinational Feedback Resolution:** The hazard was successfully mitigated by engineering an instantaneous combinational feedback loop (`next_sum`). By calculating the local running sum asynchronously within the same clock cycle, the datapath bypasses the register delay entirely.
- **Final Optimization:** While the local sum leverages combinational feedback to prevent stale reads, the global maximum comparison remains strictly registered. This hybrid architecture perfectly balances hazard-free math with high-speed pipeline clocking.

Simulation and Verification

```

sharwan@sharwan-Yoga-Slim-7-14IMH9: ~/coa_project
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$ {verilog -o sin_executable max_subarray_system_tb.v max_subarray_system.v max_subarray_pipeline.v control_fsm.v memory_rom.v
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$ vvp sin_executable
VCD info: dumpfile pipeline_waves.vcd opened for output.
.....
Simulation Complete.
Calculated Maximum Subarray Sum:      7
Expected Result: 6
.....
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$ gtkwave pipeline_waves.vcd
Gtk-Message: 20:24:15.927: Failed to load module "canberra-gtk-module"

GTKWave Analyzer v3.3.104 (w)1999-2020 BSI
[0] start time.
[195000] end time.
WM Destroy
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$ gedit max_subarray_pipeline.v
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$ {verilog -o sin_executable max_subarray_system_tb.v max_subarray_system.v max_subarray_pipeline.v control_fsm.v memory_rom.v
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$ vvp sin_executable
VCD info: dumpfile pipeline_waves.vcd opened for output.
.....
Simulation Complete.
Calculated Maximum Subarray Sum:      6
Expected Result: 6
.....
sharwan@sharwan-Yoga-Slim-7-14IMH9:~/coa_project$

```

Figure 4: Icarus Verilog Terminal Output.

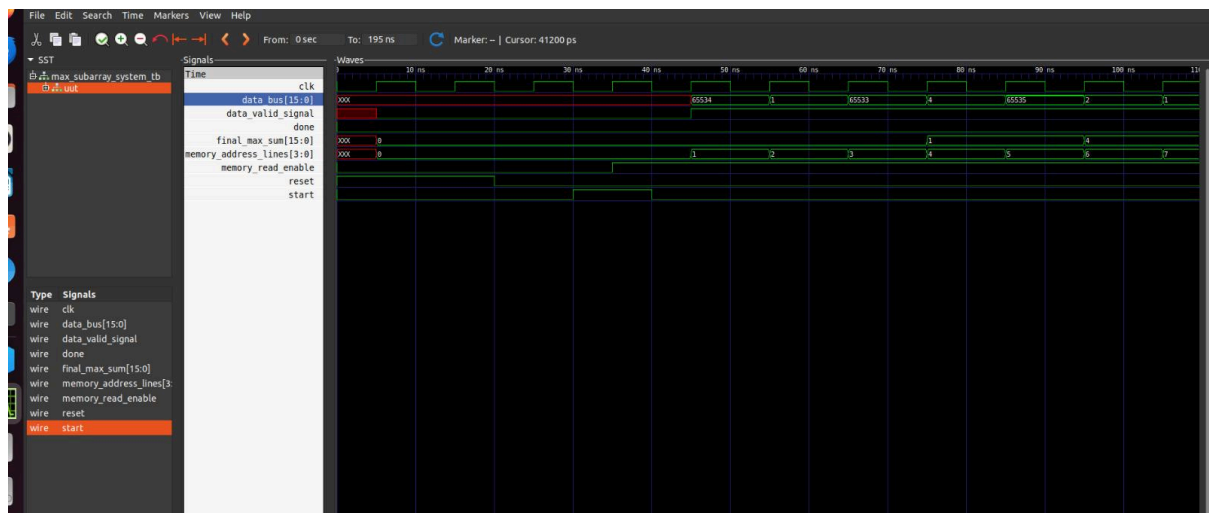


Figure 5: GTKWave Timing Waveforms and Pipeline Flush cycles.

- **Verification Toolchain:** The RTL logic was written in VS Code, synthesized and compiled using Icarus Verilog, and visually verified by rendering VCD (Value Change Dump) databases in GTKWave.
- **Test Vector Initialization:** The Memory ROM was loaded with the signed vector [-2, 1, -3, 4, -1, 2, 1, -5, 4].
- **Expected vs. Calculated Result:** The theoretical mathematical maximum of the test array is 6. The GTKWave simulation confirmed that the `final_max_sum` output bus successfully stabilized at exactly 6.
- **Cycle-by-Cycle Observation:** Waveform analysis confirmed that the FSM `memory_address` incremented precisely once per clock cycle, the `data_bus` propagated the ROM integers without dropping values, and the `done` signal remained low until the `WAIT_PIPE` flush cycles were totally completed.
- **Data Format Specification:** To accurately verify the internal math, the GTKWave data buses were configured to display in "Signed Decimal" format, properly interpreting the Two's Complement binary used by the Verilog synthesis engine for negative numbers.

Code Implementation Breakdown

- **max_subarray_system.v (Top Module):** Acts as the physical breadboard. It does not perform logic; instead, it defines the connecting wires and instantiates the FSM, ROM, and Pipeline as internal hardware components.

- **control_fsm.v (Control Unit):** Houses the combinational `case` statement that defines the state transitions, alongside the synchronous `@(posedge clk)` block that manages the actual state switching and internal counter memory.
- **memory_rom.v (Storage Unit):** Features an `initial` block that loads the pre-defined signed integer array into a synchronous register array, outputting data only when triggered by the FSM's read-enable pin.
- **max_subarray_pipeline.v (Datapath):** Contains the core logic. It utilizes signed 16-bit registers and non-blocking (`<=`) assignments to infer physical D-Flip Flops, alongside the combinational feedback logic that resolves the RAW hazard.
- **max_subarray_system_tb.v (Testbench):** Creates a dummy top-level scope. It generates a continuous 10-nanosecond clock period, sequentially drops the reset line, asserts the start line, and monitors the final output port to automatically print the calculated result to the terminal.

Conclusion

- The hardware implementation of Kadane's Algorithm was highly successful, definitively bridging the gap between high-level software theory and low-level digital logic design.
- By strictly separating the control logic from the datapath arithmetic, the resulting architecture is highly modular, easy to debug, and capable of scaling to larger memory arrays without requiring a redesign of the pipeline.
- The identification and resolution of the Read-After-Write hazard using combinational feedback highlights the critical differences between sequential software execution and parallel hardware timing.
- The final simulated waveforms prove that the pipelined architecture successfully achieves a steady-state throughput of one element per clock cycle, making it an incredibly efficient, synthesis-ready core for potential deployment onto an FPGA.

Appendix: Verilog Source Code

1. max_subarray_system.v

```
`timescale 1ns / 1ps
```

```
module max_subarray_system (  
    input clk,  
    input reset,  
    input start,  
    output signed [15:0] final_max_sum,  
    output done  
);  
  
    wire [3:0] memory_address_lines;  
    wire memory_read_enable;  
    wire signed [15:0] data_bus;  
    wire data_valid_signal;  
  
    // Instantiate ROM  
    memory_rom mem_inst (  
        .clk(clk),  
        .address(memory_address_lines),  
        .read_enable(memory_read_enable),  
        .data_out(data_bus)  
    );  
  
    // Instantiate FSM  
    control_fsm fsm_inst (  

```

```
.clk(clk),  
  
.reset(reset),  
  
.start(start),  
  
.memory_address(memory_address_lines),  
  
.memory_read_enable(memory_read_enable),  
  
.data_valid(data_valid_signal),  
  
.system_done(done)  
);  
  
  
// Instantiate Pipeline  
max_subarray_pipeline pipeline_inst (  
  
.clk(clk),  
  
.reset(reset),  
  
.data_in(data_bus),  
  
.data_valid(data_valid_signal),  
  
.max_sum(final_max_sum)  
);  
  
endmodule
```

2. control_fsm.v

```
`timescale 1ns / 1ps

module control_fsm (
    input clk,
    input reset,
    input start,
    output reg [3:0] memory_address,
    output reg memory_read_enable,
    output reg data_valid,
    output reg system_done
);

    parameter IDLE = 2'b00, READ = 2'b01, WAIT_PIPE = 2'b10, DONE = 2'b11;
    reg [1:0] state, next_state;
    reg [3:0] count;

    // State Register & Counter (Sequential Logic - Updates ONLY on Clock)
    always @(posedge clk) begin
        if (reset) begin
            state <= IDLE;
            count <= 0;
            data_valid <= 0;
        end else begin
```

```

state <= next_state;

// Only increment count on the clock edge during these states
if (state == READ || state == WAIT_PIPE) begin
    count <= count + 1;
end

// Delay data_valid by 1 cycle to match ROM read latency
data_valid <= memory_read_enable;

end

end

// Next State Logic (Combinational Logic - NO counters here!)
always @(*) begin
    // Default outputs to prevent latches
    memory_address = count;
    memory_read_enable = 0;
    system_done = 0;
    next_state = state;

    case (state)
        IDLE: begin
            if (start) next_state = READ;
        end
    end

```

```

    READ: begin
        memory_read_enable = 1;

        // Transition when we've read all 10 elements (0 to 9)
        if (count == 4'd9) next_state = WAIT_PIPE;
    end

    WAIT_PIPE: begin
        // Wait for the pipeline to finish processing the last elements
        if (count == 4'd13) next_state = DONE;
    end

    DONE: begin
        system_done = 1;
    end

endcase

end

endmodule

```

3. memory_rom.v

```

`timescale 1ns / 1ps

module memory_rom (
    input clk,
    input [3:0] address,
    input read_enable,
    output reg signed [15:0] data_out
);

```

```
// 16-element array of 16-bit signed integers
```

```
reg signed [15:0] mem [0:15];
```

```
initial begin
```

```
    mem[0] = -16'd2;
```

```
    mem[1] = 16'd1;
```

```
    mem[2] = -16'd3;
```

```
    mem[3] = 16'd4;
```

```
    mem[4] = -16'd1;
```

```
    mem[5] = 16'd2;
```

```
    mem[6] = 16'd1;
```

```
    mem[7] = -16'd5;
```

```
    mem[8] = 16'd4;
```

```
    mem[9] = 16'd0;
```

```
    // Pad the rest of the memory with zeros
```

```
    mem[10] = 16'd0; mem[11] = 16'd0; mem[12] = 16'd0;
```

```
    mem[13] = 16'd0; mem[14] = 16'd0; mem[15] = 16'd0;
```

```
end
```

```
// Synchronous read
```

```
always @(posedge clk) begin
```

```
    if (read_enable) begin
```

```
        data_out <= mem[address];
```

```
        end

    end

endmodule
```

4. max_subarray_pipeline.v

```
`timescale 1ns / 1ps

module max_subarray_pipeline (

    input clk,

    input reset,

    input signed [15:0] data_in,

    input data_valid,

    output reg signed [15:0] max_sum

);

    reg signed [15:0] current_sum;
    reg stage1_valid;

    // The explicit Pipeline Register
    reg signed [15:0] current_sum_pipeline_reg;

    // Combinational wire for instantaneous feedback (Solves the RAW hazard)
    wire signed [15:0] next_current_sum;

    assign next_current_sum = (current_sum < 0) ? data_in : (current_sum +
data_in);
```

```
always @(posedge clk) begin

    if (reset) begin

        current_sum <= 0;

        max_sum <= 0;

        current_sum_pipeline_reg <= 0;

        stage1_valid <= 0;

    end else begin

        // Pass the valid signal down the pipeline

        stage1_valid <= data_valid;

        // STAGE 1: Calculate local sum and push to pipeline register

        if (data_valid) begin

            current_sum <= next_current_sum;

            current_sum_pipeline_reg <= next_current_sum;

        end

        // STAGE 2: Global Max Check (Delayed by 1 cycle)

        if (stage1_valid) begin

            if (current_sum_pipeline_reg > max_sum) begin

                max_sum <= current_sum_pipeline_reg;

            end

        end

    end

end
```

```
    end  
endmodule
```

5. max_subarray_system_tb.v

```
`timescale 1ns / 1ps  
  
module max_subarray_system_tb;  
  
    reg clk;  
    reg reset;  
    reg start;  
    wire signed [15:0] final_max_sum;  
    wire done;  
  
    // Instantiate the Top Level System  
    max_subarray_system uut (  
        .clk(clk),  
        .reset(reset),  
        .start(start),  
        .final_max_sum(final_max_sum),  
        .done(done)  
    );
```

```
// Generate a 10ns Clock

always #5 clk = ~clk;


initial begin

    // Setup waveform dumping for GTKWave
    $dumpfile("pipeline_waves.vcd");
    $dumpvars(0, max_subarray_system_tb);


    // Initialize Inputs
    clk = 0;
    reset = 1;
    start = 0;


    // Apply Reset
    #20;
    reset = 0;


    // Trigger Start Signal
    #10;
    start = 1;
    #10;
    start = 0; // Turn off start, let FSM take over


    // Wait for system to declare it is done
```

```
wait(done == 1);

// Give it a few extra clock cycles to settle, then print result

#20;

$display("-----");

$display("Simulation Complete.");

$display("Calculated Maximum Subarray Sum: %d", final_max_sum);

$display("Expected Result: 6");

$display("-----");

$finish;

end

endmodule
```